Fast Software Control and Bounded Deliberation: Methods, Evidence, and Limits

Felipe Santibáñez-Leal

Technical report · v0.02.019, 31 October 2026 · Licence CC BY 4.0
CAOS software-control research programme · Neuraxis
Version DOI `10.5281/zenodo.23843751` · concept DOI (always latest)
`10.5281/zenodo.23765413`
Not peer reviewed. Source code, data and records: `github.com/fsantibanezleal/CAOS_Neuraxis`

Abstract

Neuraxis evaluates whether a software controller can combine fast admissible-action selection with selective delegation to slower planners. The system couples typed action contracts, a native effect broker, persistent effect records, resource revisions, policy methods, and independent task postconditions. Twelve methods are compared across twenty software-operation families, six variants, and ten seeds, for 14,400 core episodes. Five methods, including a guarded finite-state controller, solve every core episode. The proposed resource-versioned outcome-aware controller M12 solves 1,160 episodes versus 1,200 for the guarded baseline and has higher decision latency. Four component ablations match full M12 task success. A learned routing gate chooses never to defer, and XGBoost exceeds M12 on a small held-out transfer set. The evidence does not support an M12 success advantage or a measurable task-success contribution from its named components on this suite. The companion task workbench runs bounded file-analysis workflows through a deterministic verified-operation controller; it does not route these tasks through M12. The work contributes a reproducible software-control testbed, an auditable action-execution boundary, and a falsifiable evaluation of fast/slow allocation, together with a negative result identifying benchmark saturation and cancellation handling as central limitations.

Keywords: software control, action brokerage, metacontrol, selective prediction, deliberation allocation, reproducible evaluation

1 Problem and scope

Interactive software agents select actions under changing state while respecting authority, time, and effect constraints. A language model can propose a sequence, but its context-dependent generation is costly and insufficiently constrained to act as the sole executor of every low-level operation. Neuraxis studies a computational division between a fast policy that chooses among typed candidates and a slower planner consulted when expected incremental utility justifies its cost. The system records arguments, resource versions, decisions, effects, effect records, and independently evaluated outcomes.

System One and System Two are analogies motivating a fast/slow allocation question. Neuraxis is not a model of biological cognition, and no neural equivalence is claimed. The empirical question is whether a metacontroller improves outcomes or cost over competent fixed, learned, and delegated policies while retaining a broker-enforced authority boundary. Results describe a finite authored software-operations suite, not arbitrary software or human behavior.

2 Runtime and action authority

The Reflexmesh runtime represents an action as a typed candidate with a named tool, validated arguments, resource revision, and policy provenance. A policy may rank or nominate candidates; the broker admits only those satisfying the current contract and resource state. Executed effects produce effect records and append to a journal used for replay and crash recovery. Revision checks allow stale proposals to be rejected when assumptions change. The product evaluates task success from independent postconditions.

These observations are distinct. A valid effect record establishes that an effect was recorded under the runtime contract; it does not establish optimality or task correctness. Policy confidence, broker admission, effect execution, and the final postcondition must not be conflated. The same boundary supports rule-based, learned, and planner-generated proposals.

3 Operational workbench

The application separates research comparisons from bounded work on user-supplied files. Its task interface offers release verification, source auditing, and data reconciliation. A release workflow parses structured inputs, checks tabular shape and references, compares file digests with a supplied manifest, and verifies the generated artifacts. Each workflow is a registered dependency graph. The operational controller is a fixed prerequisite policy that selects the first action admitted by the current graph and runtime. Uploaded text is parsed as data and is not executed.

This task controller is distinct from M01–M12. In particular, the task interface does not invoke M12 or the learned routing gate. The Investigations view presents the measured research methods against matched benchmark records; it does not turn a replay into a fresh task execution. This separation means that the task interface demonstrates bounded software operations with persistent effect records, while the benchmark evaluates whether the proposed metacontroller improves policy outcomes. The operational workflows are not evidence of learned metacontrol performance.

The hosted service issues short-lived bearer tokens and checks ownership on run inspection, export, and deletion. It limits concurrent runs, retained run directories, input size, and retention time. Native effect records describe effects admitted and recorded by the runtime; a separate verifier checks task outputs. These properties establish bounded execution and traceability, not a general-purpose workspace, arbitrary code execution, or a task-success advantage for M12.

4 Methods

4.1 M01 and M02: guarded control

M01 is a guarded finite-state controller with explicit state rules and candidate checks. M02 is a behavior-tree controller with explicit traversal. These operationalize rule-based control and structured reactive policies through the same admissible-action interface and task postconditions. The authored task contracts and structured transitions may favor deterministic controllers, an important property of this benchmark rather than a universal result.

4.2 Established comparison methods

M03–M08 cover learned action ranking and sequential control, including logistic and boosted-tree policies, recurrent policies, and reinforcement-learning controllers. M09 is planner-based deliberation. M10 performs fixed asynchronous delegation. M11 fits a cost-sensitive deferral gate from

incremental utility. These methods run on the common core task matrix. Their training information and compute costs are not identical; the comparisons therefore describe performance under the recorded protocol rather than a compute-equivalent tournament.

4.3 Proposed method

M12 combines recurrent action preference, effect/outcome information, persistent controller state, resource-version invalidation, and computation allocation. A pending planner answer can be rejected if its resource assumptions are stale. The falsifiable prediction is that, under matched compute constraints and independently verified postconditions, M12 improves the success-cost frontier over guarded and asynchronous-planning baselines without admitting stale or unauthorized effects. A confirming experiment needs disjoint task families, meaningful state changes during pending plans, planner failures, paired outcomes, and component ablations. This report evaluates the proposal rather than presuming its success.

5 Benchmark and protocol

The core matrix has twelve methods, twenty families, six variants, and ten seeds per method-family-variant cell: $12 \times 20 \times 6 \times 10 = 14{,}400$ episodes. Families include file routing, builds, dependencies, HTTP retries, process control, conflicts, planner use, untrusted text, long-job cancellation, and crash recovery. Canonical JSON traces bind method, case, variant, seed, decisions, metrics, and postcondition outcome. A derived index records hashes and lineage.

Failures stay in denominators. Success is the number of true independent task postconditions divided by scheduled episodes. Decision latency is measured per controller decision; total wall time includes additional work and must not be substituted for policy latency. Paired comparisons match case, variant, and seed. Bootstrap intervals over seed groups can be degenerate when outcomes do not vary, and cannot establish broad generalization from repeated deterministic episodes.

Transfer compositions and routing analyses are separate populations, not part of the core count. Four M12 ablations remove memory, effects, deferral, or invalidation, with 1,200 episodes each. Their matched outcomes are compared with full M12. Task families are few, training and inference costs differ by method, and planner or environment behavior has limited variation. These design constraints bound inference.

6 Results

6.1 Core performance

Five methods solve all 1,200 core episodes, including M01. M12 solves 1,160/1,200 (96.67%), versus 1,200/1,200 for M01. Its 40 failures are all C07 long-job cancellation: four variants each have zero successes across ten seeds. This localizes the observed failure but does not identify its cause. M01

Method	Successes	Rate	p50 ms	p95 ms
M01 guarded finite-state	1200/1200	100%	0.022	0.039
M04 XGBoost	1200/1200	100%	2.633	4.017
M11 learned deferral	1200/1200	100%	1.816	3.142
M12 proposed controller	1160/1200	96.67%	18.130	158.970

Table 1: Core outcomes and recorded decision latency.

latency is 0.0219 ms at p50 and 0.0394 ms at p95; M12 is 18.13 ms and 158.97 ms respectively. These are benchmark-recorded timings, not portable hardware guarantees. M12 incurs substantially greater controller cost while achieving lower task success on this suite.

6.2 Gate, ablations, and transfer

The learned gate was fitted on 228 states with seven positive labels. Both fitted gates selected threshold 1.01, which yields never-defer behavior. M11 solves the core suite, but the gate result does not show that learned allocation beats fixed allocation.

Each ablation matches full M12 at 1,160/1,200 successes, with zero paired task-success difference over 1,200 cells. No named component demonstrates a task-success contribution here. This does not establish that the components are ineffective in every environment; effect safety and stale-plan rejection are separate outcomes that require their own measurements. Identical seed outcomes and few independent families limit statistical interpretation.

On 24 held-out composition episodes, M04 solves 24 and M12 solves 21. The sample is small and descriptive, yet its direction does not support an M12 transfer advantage. A larger held-out set with varied environment and planner conditions is needed.

7 Interpretation

The present evidence does not support the superiority prediction. M01 has higher success and much lower decision latency; M12 fails 40 cancellation episodes; all four ablations preserve its success count; the gate chooses never to defer; and the small transfer comparison favors XGBoost. The implementation exists, but the intended fast/slow allocation benefit is not demonstrated.

Typed contracts, broker authority checks, effect records, and recoverable journals provide an engineering substrate for evaluating different policies. These are runtime properties, not evidence for a cognitive architecture. Measure them through fault injection, stale-plan rejection, recovery correctness, and unauthorized-effect attempts, separately from task success.

8 Limitations and next experiments

The core benchmark is saturated for success because a guarded finite-state controller solves it completely. A useful test of adaptive deliberation needs ambiguous or changing situations where fixed rules can fail and planning can resolve uncertainty. Training, tuning, and evaluation must use disjoint task families, with the held-out set inaccessible until evaluation.

Ten seeds provide little independent evidence when the simulator is deterministic. Future protocols should vary meaningful transitions, tool behavior, network conditions, and planner outcomes by seed, and quantify uncertainty across task families. Seven positive labels are insufficient for a robust deferral gate. Collect planner cases where a slow answer changes the outcome, as well as cases where delegation is wasteful or harmful. Separate calibration, threshold selection, and final evaluation.

Diagnose C07 cancellation through transition traces: request, acknowledgement, process termination, resource revision, and final postcondition. Evaluate any repair on an untouched cancellation family. Match compute budgets and report success, decision latency, total cost, effect safety, recovery, and planner use. A positive claim should require improvement over M01 and M04 with uncertainty reflecting independent families and no loss of authority guarantees.

9 Reproducibility and data

The repository contains the design specification, method interface, case contracts, benchmark runner, per-episode traces, derived indexes, hashes, analysis scripts, and product interface. The core index contains 14,400 episodes. Scientific artifact and notebook scripts support recomputation. Frozen traces enable deterministic inspection; fresh executions measure current runtime behavior and must be labeled separately. Experiment identifiers, package versions, and source lineage accompany the data.

10 Conclusion

Neuraxis implements bounded software-action brokerage and evaluates M01 through M12 under a common task protocol. Current results do not show an advantage for M12: M01 solves every core episode at far lower decision latency, M12 fails 40 cancellations, ablations do not change success, and the small transfer comparison favors XGBoost. These negative findings motivate a variable, family-held-out evaluation. Fast/slow allocation remains a research question rather than an established capability. The separate resource-versioned metacontrol paper specifies M12 and its falsifiable allocation hypothesis, with the same benchmark findings (Santibáñez-Leal, 2026).

References

[1] Typesafe AI. Introducing System One Models and JEV. `https://typesafe.ai/blog/introducing-system-one-models-and-jev`.

[2] M. Botvinick et al. Hierarchically organized behavior and its neural foundations: a reinforcement-learning perspective. *Cognition*, 2012. doi:10.1016/j.cognition.2011.08.011.

[3] H. Mozannar and D. Sontag. Consistent estimators for learning to defer to an expert. *ICML*, 2020. `https://proceedings.mlr.press/v119/mozannar20b.html`.

[4] Y. Bai et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv:2204.05862, 2022.

[5] F. Santibáñez-Leal. Resource-Versioned Outcome-Aware Metacontrol for Software Action Systems. Preprint, 2026. doi:10.5281/zenodo.23821087.